

Essential Linux Commands

75 Commands for AI Engineering Students in 90 Minutes

File handling · Downloading · Zip/Unzip · Text processing
NLP · Data preparation · Data cleaning · Data-scientist toolbox

Ye Kyaw Thu

Lab Leader, Language Understanding Lab, Myanmar
Visiting Professor, Core-AI Lab, NECTEC, Thailand

<https://sites.google.com/site/yekyawthunlp/> | <https://github.com/ye-kyaw-thu>

Welcome!

How this session works

Format for every command

- ❶ **Short explanation** – what the command is for
- ❷ **Basic syntax** – the general pattern
- ❸ **Common options** – the flags you'll actually use
- ❹ **Example** – a real, runnable command

Pace

75 commands, roughly **1 minute per slide**. Don't worry about memorising everything today – the slide file is yours to keep and search later.

Why the terminal, if you're an AI engineer?

Because nearly every real project – downloading a dataset, cleaning a corpus, checking GPU memory, launching a training job on a remote server – happens faster, and more reproducibly, from the command line than from a mouse.

Today's 11 Categories

Roadmap – 75 commands in 90 minutes

#	Category	Commands	Time
1	Filesystem Navigation	7	7 min
2	File & Directory Operations	8	8 min
3	Disk Usage & Permissions	6	6 min
4	Downloading & Remote Access	7	7 min
5	Compression & Archiving	5	5 min
6	Viewing & Comparing Files	6	6 min
7	Core Text Processing	10	10 min
8	Pipes, Redirection & Shell Utilities	8	8 min
9	Data Preparation & Cleaning	9	9 min
10	Process, Resource & GPU Monitoring	6	6 min
11	Environment & Version Control	3	3 min
Total		75	75 min

Based on and extending *Hostinger's "60 essential Linux commands"* and *TecMint's "40+ Linux Commands for Every Machine Learning Engineer"* – regrouped and expanded specifically for NLP and data-science workflows.

Category 1 of 11

Filesystem Navigation

7 commands (~7 min)



pwd

Category 1 – Filesystem Navigation (1/7)

Prints the full path of your **current working directory** – “where am I right now?”

Syntax

```
pwd [options]
```

Example

```
pwd
```

Common options

- -L – show the logical path (through symlinks)
- -P – show the physical, real path

Prints something like /home/aiuser/projects/nlp-lab – useful before running a script that depends on relative paths.

ls

Category 1 – Filesystem Navigation (2/7)

Lists the **contents of a directory** – files and sub-folders.

Syntax

```
ls [options] [path]
```

Example

```
ls -lh ~/datasets/
```

Common options

- `-l` – long listing (permissions, size, date)
- `-a` – show hidden files (starting with `.`)
- `-h` – human-readable sizes (KB/MB/GB)
- `-R` – recurse into sub-directories

Shows a detailed, human-readable listing of your datasets folder – sizes, permissions, and last-modified dates for every file.

Changes your current working directory – how you move around the filesystem.

Syntax

```
cd [path]
```

Example

```
cd ~/projects/nlp-lab/corpus
```

Handy shortcuts

- `cd` – go to your home directory
- `cd ..` – go up one directory
- `cd -` – go back to the previous directory

Jumps directly into a corpus folder nested several levels under your home directory.

mkdir

Category 1 – Filesystem Navigation (4/7)

Creates one or more new, empty directories.

Syntax

```
mkdir [options] dir1 dir2 ...
```

Common options

- `-p` – create parent directories as needed
- `-m` – set permissions while creating

Example

```
mkdir -p experiments/run1/{logs,checkpoints}
```

Creates `experiments/run1/logs` and `experiments/run1/checkpoints` in one shot – `-p` builds every missing parent folder along the way.

rmmdir

Category 1 – Filesystem Navigation (5/7)

Removes a directory, but **only if it is empty** – a safety-first deletion tool.

Syntax

```
rmmdir [options] dir_name
```

Example

```
rmmdir old_experiment/
```

Common options

- `-p` – also remove now-empty parent directories
- `-v` – verbose, print what was removed

Deletes `old_experiment/` only if it contains nothing – use `rm -r` instead (Category 2) if it still has files inside.

Displays a directory's contents as an indented, visual **tree** – much easier to scan than a flat `ls` listing for nested project folders.

Syntax

```
tree [options] [path]
```

Example

```
tree -L 2 ~/projects/nlp-lab
```

Common options

- `-L n` – limit depth to `n` levels
- `-d` – show directories only
- `-a` – include hidden files

Prints a two-level-deep tree of your project – a quick way to show a new student the shape of a codebase or dataset folder.

find

Category 1 – Filesystem Navigation (7/7)

Searches for files and directories by name, type, size, or modification time – searches in real time, directly through the filesystem.

Syntax

```
find [path] [options] expr
```

Example

```
find . -name "*.json" -type f -size +50M
```

Common options

- `-name` – match by filename (supports `*`)
- `-type f/d` – files only / directories only
- `-mtime -n` – modified in the last `n` days
- `-size +100M` – larger than 100 MB

Finds every JSON file larger than 50 MB under the current directory – handy for spotting oversized dataset files before a Git commit.

Category 2 of 11

File & Directory Operations

8 commands (~8 min)



touch

Category 2 – File & Directory Operations (1/8)

Creates a new, **empty file**, or updates the modification timestamp of an existing one.

Syntax

```
touch [options] file_name
```

Example

```
touch README.md .gitkeep
```

Common options

- **-a** – update access time only
- **-m** – update modification time only
- **-c** – do not create the file if missing

Creates two empty files at once – a common way to reserve a placeholder file, like keeping an empty folder tracked in Git.

Copies files or directories to a new location, leaving the original intact.

Syntax

```
cp [options] source target
```

Example

```
cp -r raw_data/ backup/raw_data_2026-07-20/
```

Common options

- **-r** – copy directories recursively
- **-v** – verbose, print each file copied
- **-u** – copy only when source is newer

Makes a dated backup copy of an entire raw-data folder – a quick habit before running a destructive cleaning script.

Moves a file or directory to a new location – also used to **rename** files, since renaming is just moving within the same folder.

Syntax

```
mv [options] source target
```

Example

```
mv model_v1.pt model_v1_baseline.pt
```

Common options

- **-i** – ask before overwriting
- **-n** – never overwrite an existing file
- **-v** – verbose output

Renames a saved model checkpoint to a clearer, more descriptive filename before starting a new round of experiments.

Deletes files or directories permanently – there is no Recycle Bin in the terminal, so use it carefully.

Syntax

```
rm [options] file1 file2 ...
```

Example

```
rm -ri old_checkpoints/
```

Common options

- **-r** – remove directories and their contents
- **-f** – force, no confirmation prompts
- **-i** – ask for confirmation before each deletion

*Deletes an old checkpoints folder, asking for confirmation first – much safer than **-rf** when clearing out large training artefacts.*

Creates a **link** to a file – most commonly a **symbolic link** (shortcut) that points to a file living somewhere else.

Syntax

```
ln [options] source target
```

Example

```
ln -s /data/shared/wikipedia_corpus ./corpus
```

Common options

- `-s` – create a symbolic (soft) link
- `-f` – overwrite an existing link

Creates a shortcut named `corpus` in your project folder pointing to a large shared dataset – avoids duplicating gigabytes of data on disk.

file

Category 2 – File & Directory Operations (6/8)

Identifies a file's **type** by inspecting its content, not just its extension – useful when a downloaded file's name lies about what it actually is.

Syntax

```
file [options] file_name
```

Example

```
file dataset.bin
```

Common options

- `-i` – show the MIME type
- `-z` – inspect inside compressed files too

Reports the real format of `dataset.bin` – e.g. “gzip compressed data” – so you know whether to `gunzip` it first or load it directly.

Displays **detailed metadata** about a file: exact size in bytes, permissions, owner, and precise access/modification timestamps.

Syntax

```
stat [options] file_name
```

Example

```
stat -c "%s bytes, modified %y" train.csv
```

Common options

- `-c` – custom output format
- `--printf` – print without a trailing newline

Prints the exact byte size and last-modified time of a training CSV – more precise than `ls -l` when scripting checks.

locate

Category 2 – File & Directory Operations (8/8)

Searches a pre-built **system database** for file paths matching a keyword – much faster than `find`, but the database must be kept up to date.

Syntax

```
locate [options] keyword
```

Example

```
sudo updatedb && locate -i "bert*.json"
```

Common options

- `-i` – case-insensitive search
- `-c` – print a count instead of paths

Refreshes the search database, then instantly finds every file whose name matches `bert*.json` anywhere on the system.

Category 3 of 11

Disk Usage & Permissions

6 commands (~6 min)



Reports how much **disk space** a file or directory (and its contents) is using – essential for hunting down what's filling up your drive.

Syntax

```
du [options] [path]
```

Example

```
du -sh datasets/* | sort -rh | head -n 10
```

Common options

- `-s` – summarise, one total per argument
- `-h` – human-readable sizes (KB/MB/GB)
- `-a` – show sizes for individual files too

Lists the 10 largest items inside `datasets/` – the go-to command when a shared server is running low on disk space.

df

Category 3 – Disk Usage & Permissions (2/6)

Shows how much space is used and free on each **mounted filesystem** (whole disks/partitions), not individual folders.

Syntax

```
df [options] [filesystem]
```

Example

```
df -h
```

Common options

- `-h` – human-readable sizes
- `-T` – show filesystem type

Shows total, used, and available space on every mounted drive – run this before starting a large model download or dataset export.

chmod

Category 3 – Disk Usage & Permissions (3/6)

Changes permissions on a file or directory – who can read (r), write (w), or execute (x) it.

Syntax

```
chmod [options] mode file
```

Example

```
chmod +x train.sh && ./train.sh
```

Common options

- `+x` – add execute permission
- `-w` – remove write permission
- `755` – owner: `rwX`, others: `r-X` (numeric form)

Makes a training shell script executable, then runs it – you'll hit "Permission denied" the first time you try this without `+x`.

chown

Category 3 – Disk Usage & Permissions (4/6)

Changes the **owner** (and optionally group) of a file or directory – requires administrator privileges for files you don't already own.

Syntax

```
chown [options] user:group file
```

Example

```
sudo chown -R aiuser:aiteam shared_project/
```

Common options

- **-R** – apply recursively to a whole directory
- **-v** – verbose output

Hands ownership of an entire shared project folder over to a specific user and team group – common after copying files in as root.

sudo

Category 3 – Disk Usage & Permissions (5/6)

“superuser do” – runs a single command with administrator privileges, for actions a normal user isn't allowed to do.

Syntax

```
sudo [options] command
```

Example

```
sudo apt install nvidia-cuda-toolkit
```

Common options

- `-i` – start an interactive root shell
- `-u user` – run as a specific user, not root

Installs a system-wide package that requires administrator rights – you'll be prompted once for your password.

whoami

Category 3 – Disk Usage & Permissions (6/6)

Prints the **username** you are currently logged in as – a fast sanity check, especially on a shared remote server.

Syntax

```
whoami
```

Example

```
whoami
```

Related commands

- `id` – show user and group IDs
- `who` – list everyone logged in

Prints e.g. `aiuser` – worth checking right after SSH-ing into a shared GPU server, so you don't accidentally run a job as the wrong account.

Category 4 of 11

Downloading & Remote Access

7 commands (~7 min)



wget

Category 4 – Downloading & Remote Access (1/7)

Downloads files from the web over HTTP, HTTPS, or FTP – simple, scriptable, and great for grabbing datasets or model files by direct URL.

Syntax

```
wget [options] URL
```

Example

```
wget -c https://example.com/imdb_reviews.zip
```

Common options

- `-O` – save with a custom filename
- `-c` – resume an interrupted download
- `-P` – choose the destination folder

Downloads a dataset archive, and `-c` means if your connection drops halfway, re-running the command resumes rather than restarting.

curl

Category 4 – Downloading & Remote Access (2/7)

Transfers data to or from a URL – like `wget`, but also excellent for **testing APIs** (REST endpoints, model-serving servers).

Syntax

```
curl [options] URL
```

Example

```
curl -X POST -H "Content-Type: application/json" \  
-d '{"text":"hello"}' http://localhost:8000/predict
```

Common options

- `-O` – save output using the remote filename
- `-X` – set the HTTP method (GET/POST/...)
- `-H` – add a request header

Sends a JSON request to a locally running model API – exactly how you'd smoke-test an inference server before deploying it.

ssh

Category 4 – Downloading & Remote Access (3/7)

Secure Shell – opens an encrypted terminal session on a remote machine, such as a cloud GPU server.

Syntax

```
ssh [options] user@host
```

Example

```
ssh -i ~/.ssh/gpu_key.pem ubuntu@gpu-server.edu
```

Common options

- `-p` – connect on a specific port
- `-i` – use a specific private key file

Logs into a remote GPU training server using a private key – the standard first step of almost every cloud ML workflow.

Secure copy – transfers files or folders between machines over SSH's encrypted connection.

Syntax

```
scp [options] source target
```

Example

```
scp results.csv ubuntu@gpu-server.edu:~/outputs/
```

Common options

- **-r** – copy directories recursively
- **-P** – specify the SSH port
- **-C** – enable compression during transfer

Copies a local results file up to a folder on the remote GPU server – no need to open a separate FTP client.

rsync

Category 4 – Downloading & Remote Access (5/7)

Synchronises files between two locations, transferring only what changed – much faster than scp for large, repeated transfers.

Syntax

```
rsync [options] source dest
```

Example

```
rsync -avz ./checkpoints/ user@server:/backup/checkpoints/
```

Common options

- `-a` – archive mode (preserves structure, permissions)
- `-v` – verbose
- `-z` – compress data during transfer

Keeps a remote backup of your model checkpoints in sync – re-running it later only sends the newly changed files, saving time.

ping

Category 4 – Downloading & Remote Access (6/7)

Sends test packets to a host to check whether it's **reachable** and to measure round-trip response time – the first step of network troubleshooting.

Syntax

```
ping [options] host
```

Example

```
ping -c 4 8.8.8.8
```

Common options

- `-c n` – send exactly `n` packets and stop
- `-i` – set interval between packets

Sends 4 packets to Google's DNS server – a quick way to confirm your training server actually has internet access before a big download.

git clone

Category 4 – Downloading & Remote Access (7/7)

Downloads a complete copy of a **Git repository** – code, history, and branches – from a remote host such as GitHub.

Syntax

```
git clone [options] URL
```

Example

```
git clone https://github.com/ye-kyaw-thu/nlp-tools.git
```

Common options

- `--depth 1` – shallow clone, history-free (faster)
- `-b` – clone a specific branch

Downloads a full copy of an NLP tools repository into a new local folder, ready to explore, run, or contribute to.

Category 5 of 11

Compression & Archiving

5 commands (~5 min)



tar

Category 5 – Compression & Archiving (1/5)

Bundles many files and folders into a single **archive** file – optionally compressed at the same time. The classic “.tar.gz” you see everywhere.

Syntax

```
tar [options] archive.tar files
```

Example

```
tar -czvf corpus_backup.tar.gz ./corpus/
```

Common options

- -c – create a new archive
- -x – extract an archive
- -z – also gzip-compress
- -v – verbose, list files processed
- -f – specify the archive filename

Bundles and compresses an entire corpus folder into one file – the standard way to package a dataset for sharing or archiving.

Compresses files into a **.zip** archive – the most universally compatible format, easy to open on Windows, macOS, and Linux alike.

Syntax

```
zip [options] archive.zip files
```

Example

```
zip -r results_2026.zip results/ report.pdf
```

Common options

- **-r** – recurse into directories
- **-9** – maximum compression level
- **-e** – encrypt with a password

Packages a results folder and a report together into one zip file – handy when submitting an assignment or sharing results by email.

unzip

Category 5 – Compression & Archiving (3/5)

Extracts the contents of a **.zip** archive back into individual files and folders.

Syntax

```
unzip [options] archive.zip
```

Example

```
unzip -d ./data glove_embeddings.zip
```

Common options

- `-d` – extract into a specific directory
- `-l` – list contents without extracting
- `-o` – overwrite existing files without asking

Extracts a downloaded word-embeddings archive straight into a `data` folder – exactly what you do right after `wget`-ing a dataset.

Compresses a **single file** in place, producing a `.gz` file – extremely common for compressing large text corpora and log files.

Syntax

```
gzip [options] file_name
```

Example

```
gzip -k train_corpus.txt
```

Common options

- `-k` – keep the original file too
- `-9` – maximum compression
- `-v` – show the compression ratio achieved

Compresses a large plain-text training corpus to `train_corpus.txt.gz` while keeping the original – many NLP tools can even read `.gz` files directly, saving disk space permanently.

gunzip

Category 5 – Compression & Archiving (5/5)

Decompresses a .gz file back to its original, uncompressed form – the exact reverse of gzip.

Syntax

```
gunzip [options] file.gz
```

Example

```
gunzip -c reviews.txt.gz | head -n 5
```

Common options

- -k – keep the .gz file too
- -c – decompress to standard output (don't write a file)

Decompresses straight to the screen and pipes just the first 5 lines to head – peek inside a compressed corpus without ever writing an uncompressed copy to disk.

Category 6 of 11

Viewing & Comparing Files

6 commands (~6 min)



Concatenates and prints file contents to the screen – the simplest way to dump a whole (short) file, or to join several files together.

Syntax

```
cat [options] file1 file2 ...
```

Example

```
cat train_part1.txt train_part2.txt > train_full.txt
```

Common options

- **-n** – number every output line
- **-A** – show hidden characters (tabs, line ends)

Joins two corpus shards into one combined training file – a very common data-preparation step before tokenising a whole dataset.

less

Category 6 – Viewing & Comparing Files (2/6)

Opens a file for **scrollable viewing**, one screen at a time, without loading the whole thing into memory – ideal for huge log or corpus files.

Syntax

```
less [options] file_name
```

Example

```
less -N training.log
```

Keys while inside less

- /word – search forward for “word”
- space – next page, q – quit

Opens a large training log with line numbers shown – you can scroll and search through gigabytes of logs without them ever fully loading into memory.

head

Category 6 – Viewing & Comparing Files (3/6)

Prints just the **first few lines** of a file – the fastest way to peek at a dataset's structure without opening the whole thing.

Syntax

```
head [options] file_name
```

Example

```
head -n 3 train.csv
```

Common options

- `-n N` – show the first N lines
- `-c N` – show the first N bytes

Shows the header row and first two data rows of a CSV – always the first command to run on a new, unfamiliar dataset file.

tail

Category 6 – Viewing & Comparing Files (4/6)

Prints just the **last few lines** of a file – and, with `-f`, can **follow** a file live as new lines are appended.

Syntax

```
tail [options] file_name
```

Example

```
tail -f training.log
```

Common options

- `-n N` – show the last N lines
- `-f` – follow the file live as it grows

Streams new lines from a training log in real time as your model trains – the standard way to watch loss values update live.

Word count – counts lines, words, and characters/bytes in a file. For NLP work, “lines” very often means “sentences”.

Syntax

```
wc [options] file_name
```

Example

```
wc -l train.txt dev.txt test.txt
```

Common options

- `-l` – count lines only
- `-w` – count words only
- `-c` – count bytes; `-m` – count characters

Reports how many sentences are in each split of your dataset – the very first sanity check before training any NLP model.

diff

Category 6 – Viewing & Comparing Files (6/6)

Compares two files **line by line** and reports exactly what differs – essential for checking what changed between two versions of code or data.

Syntax

```
diff [options] file1 file2
```

Example

```
diff -u config_v1.yaml config_v2.yaml
```

Common options

- `-u` – unified diff format (easier to read)
- `-q` – just report whether files differ
- `-r` – compare directories recursively

Shows exactly which hyperparameters changed between two experiment config files – much faster than eyeballing two files side by side.

Category 7 of 11

Core Text Processing

10 commands (~10 min)



Searches text for lines matching a pattern (plain text or a regular expression) – arguably the single most-used command in this whole deck.

Syntax

```
grep [options] pattern file
```

Example

```
grep -ri "cuda out of memory" logs/*.log
```

Common options

- **-i** – case-insensitive
- **-r** – recurse through directories
- **-v** – invert match (show non-matching lines)
- **-c** – count matching lines instead of printing them

Searches every log file for a common GPU error message, case-insensitively – the fastest way to find why a training run crashed.

Stream editor – transforms text on the fly, most famously for find-and-replace, without ever opening the file in an editor.

Syntax

```
sed [options] 'script' file
```

Example

```
sed -i 's/<br \/>/ /g' reviews.txt
```

Common options

- `s/old/new/` – replace first match per line
- `s/old/new/g` – replace all matches per line
- `-i` – edit the file in place

Strips leftover HTML line-break tags out of a scraped text corpus, in place – a classic text-cleaning step before tokenisation.

A full pattern-scanning and column-processing language – perfect for working with structured text like CSV/TSV files, column by column.

Syntax

```
awk 'pattern {action}' file
```

Example

```
awk -F',' '{print $1, $NF}' data.csv
```

Common options

- `-F` – set the field (column) separator
- `$1`, `$2`, ... – refer to columns
- `$NF` – the last column

Prints just the first and last column of a CSV file – extremely handy for quickly extracting one or two fields from tabular data.

sort

Category 7 – Core Text Processing (4/10)

Sorts the lines of a file **alphabetically or numerically** – almost always the first half of a “sort, then count” data-analysis pipeline.

Syntax

```
sort [options] file
```

Example

```
sort -k2 -n -r scores.tsv | head -n 5
```

Common options

- `-n` – sort numerically
- `-r` – reverse (descending) order
- `-u` – remove duplicate lines while sorting
- `-k` – sort by a specific column

Sorts a tab-separated file by its 2nd column, largest first, and shows the top 5 – e.g. the 5 highest-scoring model predictions.

Collapses **consecutive** duplicate lines into one – almost always used right after sort, since `uniq` only sees neighbours.

Syntax

```
uniq [options] file
```

Example

```
sort words.txt | uniq -c | sort -rn | head -n 20
```

Common options

- `-c` – prefix each line with its count
- `-d` – show only lines that had duplicates
- `-u` – show only lines that were unique

The classic Linux word-frequency pipeline: sort words, collapse duplicates into counts, then sort by frequency – the top 20 most common words in a corpus, in one line.

cut

Category 7 – Core Text Processing (6/10)

Extracts specific **columns or character ranges** from each line of text – a lightweight alternative to `awk` for simple column selection.

Syntax

```
cut [options] file
```

Example

```
cut -d',' -f2,4 metadata.csv
```

Common options

- `-d` – set the field delimiter
- `-f` – select which field(s) to keep
- `-c` – select by character position

Extracts just the 2nd and 4th comma-separated columns from a metadata file – e.g. pulling out `label` and `file_path` columns only.

Translates or deletes individual characters from text – great for quick case conversion or stripping unwanted symbols.

Syntax

```
tr [options] set1 set2
```

Example

```
cat text.txt | tr '[:upper:]' '[:lower:]'
```

Common options

- `-d` – delete the characters in `set1`
- `-s` – squeeze repeated characters into one
- `[:upper:]` `[:lower:]` – case classes

Lower-cases an entire text file – a very common text-normalisation step before building a vocabulary for an NLP model.

Merges corresponding lines from multiple files **side by side**, column by column – perfect for pairing parallel text data.

Syntax

```
paste [options] file1 file2
```

Example

```
paste -d'\t' source.txt target.txt > parallel.tsv
```

Common options

- `-d` – set the delimiter between columns
- `-s` – merge lines into a single row instead

Combines a source-language file and target-language file into one tab-separated parallel corpus – exactly the format machine translation training data needs.

join

Category 7 – Core Text Processing (9/10)

Merges two **sorted** files based on a shared key column – like a database join, right there in the shell.

Syntax

```
join [options] file1 file2
```

Example

```
join -t',' -1 1 -2 1 ids_labels.csv ids_scores.csv
```

Common options

- -1, -2 – key field number in each file
- -t – set the field separator

Joins a labels file and a scores file on a shared ID column – both files must be sorted by that key column first.

Reverses the **characters** of each line – a small utility, but genuinely useful for certain string-manipulation and text-processing tricks.

Syntax

```
rev file_name
```

Example

```
echo "hello world" | rev
```

Common pairing

- combine with cut to select fields counted from the *end* of irregular-length lines

Prints `dlrow olleh` – a small building block that occasionally solves an otherwise fiddly text-parsing problem in one line.

Category 8 of 11

Pipes, Redirection & Shell Utilities

8 commands (~8 min)



Prints text (or the value of a variable) to the screen – the simplest building block for shell scripts and quick sanity checks.

Syntax

```
echo [options] "text"
```

Common options

- `-n` – no trailing newline
- `-e` – interpret backslash escapes (`\n`)

Example

```
echo "Training started at $(date)" >> run.log
```

Appends a timestamped line to a log file – a simple habit that makes debugging a long training run much easier afterwards.

printf

Category 8 – Pipes, Redirection & Shell Utilities (2/8)

Prints **formatted** text – like echo, but with precise control over number formatting, padding, and spacing.

Syntax

```
printf "format" args
```

Example

```
printf "Epoch %d: acc=%.2f%%\n" 5 91.837
```

Common options

- %s – string; %d – integer
- %.2f – float with 2 decimal places

Prints Epoch 5: acc=91.84% – precise numeric formatting that plain echo can't easily do, handy inside training scripts.

xargs

Category 8 – Pipes, Redirection & Shell Utilities (3/8)

Takes input piped from another command and **builds and runs** a new command with it as arguments – the glue that connects many pipelines together.

Syntax

```
cmd1 | xargs [options] cmd2
```

Example

```
find . -name "*.tmp" | xargs rm
```

Common options

- `-n N` – pass N arguments per call
- `-I {}` – placeholder for each input item
- `-P` – run calls in parallel

Finds every temporary file and deletes them all in one go – `find` alone only lists files, `xargs` is what actually turns that list into action.

Splits output so it goes **both** to the screen *and* into a file at the same time – named after a plumbing T-pipe fitting.

Syntax

```
cmd | tee [options] file
```

Example

```
python train.py | tee -a training.log
```

Common options

- `-a` – append instead of overwrite

Lets you watch your model train live in the terminal while simultaneously saving every line to a log file for later inspection.

history

Category 8 – Pipes, Redirection & Shell Utilities (5/8)

Shows the list of **previously run commands** in your shell session – great for recalling that one complicated command from earlier.

Syntax

```
history [N]
```

Example

```
history | grep "python train"
```

Handy tricks

- `!123` – re-run history item number 123
- `Ctrl+R` – interactive reverse search

Searches your command history for every past training run you launched – much faster than trying to remember the exact flags you used last week.

Creates a **shortcut name** for a longer command – a small quality-of-life tool that saves a lot of repetitive typing.

Syntax

```
alias name='command'
```

Notes

- add to ~/.bashrc to make it permanent
- unalias name removes it

Example

```
alias gpu='watch -n 1 nvidia-smi'
```

Now typing just `gpu` refreshes a live GPU-usage display every second – a nice example of turning a long command into a one-word habit.

Opens the **manual page** for a command – the official, built-in documentation, always available even without internet access.

Syntax

```
man command_name
```

Example

```
man sort
```

Inside man

- /word – search; q – quit
- -k keyword – search manual page titles

Opens the full official documentation for `sort` – whenever you forget a flag from today's slides, this is the ultimate source of truth.

watch

Category 8 – Pipes, Redirection & Shell Utilities (8/8)

Re-runs a command **repeatedly** at a fixed interval, refreshing the screen – perfect for monitoring something that changes over time.

Syntax

```
watch [options] 'command'
```

Example

```
watch -n 2 'ls -lh checkpoints/ | tail -n 5'
```

Common options

- `-n N` – refresh every N seconds
- `-d` – highlight what changed between runs

Refreshes a view of your 5 newest model checkpoints every 2 seconds – a simple live dashboard for a long training job, no extra software required.

Category 9 of 11

Data Preparation & Cleaning

9 commands (~9 min)



split

Category 9 – Data Preparation & Cleaning (1/9)

Splits one large file into several **smaller pieces** – by line count or by size. Ideal for sharding a huge corpus for parallel processing.

Syntax

```
split [options] file prefix
```

Example

```
split -l 100000 -d corpus.txt shard_
```

Common options

- `-l N` – split every N lines
- `-n N` – split into N equal chunks
- `-d` – use numeric suffixes

Breaks a huge corpus into 100,000-line shards named `shard_00`, `shard_01`, ... – ready for distributed preprocessing across multiple machines or cores.

Randomly shuffles the lines of a file – essential for removing ordering bias before splitting data into train/dev/test sets.

Syntax

```
shuf [options] file
```

Example

```
shuf dataset.tsv -o dataset_shuffled.tsv
```

Common options

- `-n N` – output only N random lines
- `-o` – write result to a file
- `--random-source` – reproducible shuffling

Randomises row order before a train/dev/test split – without this, a dataset sorted by class or date can badly bias your splits.

Compares two **sorted** files line by line and reports what's unique to each, and what's shared – great for comparing vocabulary lists.

Syntax

```
comm [options] file1 file2
```

Example

```
comm -12 vocab_train.txt vocab_test.txt
```

Common options

- -1 – suppress lines unique to file1
- -2 – suppress lines unique to file2
- -3 – suppress lines common to both

Shows only the words that appear in both the training and test vocabularies – useful for checking vocabulary overlap and out-of-vocabulary rates.

Numbers the lines of a file – similar to `cat -n`, but with more control over the numbering format.

Syntax

```
nl [options] file
```

Common options

- `-w` – width of the number field
- `-s` – separator between number and text

Example

```
nl -s ': ' sentences.txt > sentences_ids.txt
```

Adds a sentence-ID prefix to every line of a corpus – handy for tracking which model prediction corresponds to which original sentence.

Converts text between **character encodings** – essential when a scraped or donated corpus isn't in the UTF-8 your NLP tools expect.

Syntax

```
iconv -f enc1 -t enc2 file
```

Example

```
iconv -f latin1 -t utf-8 legacy.txt > clean_utf8.txt
```

Common options

- -f – source encoding
- -t – target encoding
- -l – list all supported encodings

Converts an older Latin-1-encoded text file to clean UTF-8 – a very common fix when a multilingual corpus produces garbled characters ("mojibake") in your tokenizer.

Converts Windows-style line endings (CRLF) to Unix-style (LF) – fixes a very common, very quiet source of data-cleaning bugs.

Syntax

```
dos2unix [options] file
```

Example

```
dos2unix train.tsv
```

Common options

- `-k` – keep the original timestamp
- `unix2dos` – the reverse conversion

Fixes a dataset exported from Excel or Windows, where invisible `^M` characters at line ends can silently break line counts and tokenisers.

md5sum

Category 9 – Data Preparation & Cleaning (7/9)

Computes a short **fingerprint (checksum)** of a file's contents – used to verify a dataset downloaded correctly, byte for byte.

Syntax

```
md5sum [options] file
```

Common options

- `-c` – check files against a saved checksum list

Example

```
md5sum dataset.zip
```

Prints a fingerprint you can compare against the value published by the dataset provider – confirms your download wasn't corrupted or tampered with.

sha256sum

Category 9 – Data Preparation & Cleaning (8/9)

Like md5sum, but computes a longer, more collision-resistant **SHA-256** fingerprint – the modern standard for verifying file integrity.

Syntax

```
sha256sum [options] file
```

Example

```
sha256sum -c checksums.sha256
```

Common options

- `-c` – check against a saved checksum list

Verifies an entire folder of downloaded model weights against a published checksum file in one command – standard practice before trusting a pretrained model file.

A dedicated **JSON processor** – filters, extracts, and reformats JSON data directly in the terminal. Invaluable for API responses and JSON datasets.

Syntax

```
jq [options] 'filter' file.json
```

Example

```
jq -r '.[ ] | .text' reviews.json > reviews.txt
```

Common options

- `.field` – extract one field
- `-r` – raw output (no quotes)
- `. [ ]` – iterate over an array

Extracts just the `text` field from every record in a JSON dataset into a plain-text file – a very common first step in an NLP data-preparation pipeline.

Category 10 of 11

Process, Resource & GPU Monitoring

6 commands (~6 min)



Lists currently **running processes** – a snapshot of what's using your CPU and memory right now.

Syntax

```
ps [options]
```

Example

```
ps aux | grep python
```

Common options

- `aux` – show all processes, all users
- `-ef` – full-format listing (alternate style)

Lists every running Python process – useful for finding the process ID of a training job you need to check on or stop.

Shows a **live, updating** view of running processes, sorted by resource usage – the built-in real-time task manager.

Syntax

```
top [options]
```

Example

```
top
```

Keys while inside top

- P – sort by CPU usage
- M – sort by memory usage
- q – quit

Opens a live dashboard of CPU and memory usage – the first thing to check when a server “feels slow” during a training run.

htop

Category 10 – Process, Resource & GPU Monitoring (3/6)

An improved, colourised, mouse-friendly version of `top` – easier to read at a glance (may need a quick `sudo apt install htop` first).

Syntax

```
htop [options]
```

Example

```
htop
```

Keys while inside htop

- F6 – change sort column
- F9 – kill the selected process

Same job as `top`, but with per-core CPU bars and a visual process tree – most engineers install this as their daily driver.

kill

Category 10 – Process, Resource & GPU Monitoring (4/6)

Sends a signal to a running process, most commonly to **stop** it – identified by its process ID (PID) from `ps` or `top`.

Syntax

```
kill [options] PID
```

Example

```
kill -9 24817
```

Common options

- `-9` – force kill (SIGKILL, immediate)
- `-15` – polite request to stop (SIGTERM, default)
- `killall name` – kill by process name

Force-stops a runaway training process that's stuck and not responding to a normal stop request – use `-9` as a last resort.

Shows how much **RAM** (and swap space) is currently used and available on the system.

Syntax

```
free [options]
```

Example

```
free -h
```

Common options

- **-h** – human-readable sizes
- **-s N** – refresh every N seconds

Shows total, used, and free memory in GB – check this before loading a huge dataset entirely into memory, to avoid crashing the machine.

nvidia-smi

Category 10 – Process, Resource & GPU Monitoring (6/6)

NVIDIA System Management Interface – shows GPU utilisation, memory usage, temperature, and which processes are using each GPU.

Syntax

```
nvidia-smi [options]
```

Common options

- `-l N` – refresh every N seconds
- `--query-gpu` – select specific fields

Example

```
watch -n 1 nvidia-smi
```

Live-updates GPU memory and utilisation every second – the single most-used command when debugging “CUDA out of memory” errors during deep-learning training.

Category 11 of 11

Environment & Version Control

3 commands (~3 min)



Python's package installer – installs, upgrades, and removes Python libraries like `numpy`, `torch`, or `transformers`.

Syntax

```
pip install [options] package
```

Example

```
pip install -r requirements.txt
```

Common options

- `-r requirements.txt` – install a whole list
- `--upgrade` – upgrade to the latest version
- `freeze` – list installed packages & versions

Installs every package listed in a project's requirements file in one command – the standard way to set up someone else's (or your own past) project reproducibly.

A package *and* environment manager – creates isolated Python environments so different projects' dependencies never conflict.

Syntax

```
conda [command] [options]
```

Example

```
conda create -n nlp python=3.11 && conda activate nlp
```

Common options

- `create -n name python=3.11` – new environment
- `activate name` – switch into it
- `env list` – show all environments

Creates a clean, isolated environment called `nlp` and switches into it – keeps this course's packages completely separate from any other project on your machine.

git status

Category 11 – Environment & Version Control (3/3)

Shows which files in a Git repository are **changed, staged, or untracked** – the command you run constantly while working with Git.

Syntax

```
git status [options]
```

Example

```
git status
```

The core Git loop

- `git add file` – stage a change
- `git commit -m "msg"` – save a snapshot
- `git push` – upload to GitHub

Shows exactly what's changed since your last commit – run this before every `git add` so you always know what you're about to save.

Putting It All Together

Closing – one pipeline using many of today's commands

A realistic mini NLP data-prep pipeline, combining commands from several categories you just learned:

A real NLP data-preparation pipeline

```
wget -c https://example.com/reviews.zip
unzip reviews.zip && rm reviews.zip
jq -r '.[ ] | .text' reviews.json > reviews.txt
sed -i 's/<br \\/>/ /g' reviews.txt
tr '[:upper:]' '[:lower:]' < reviews.txt > reviews_clean.txt
shuf reviews_clean.txt -o reviews_shuffled.txt
split -n 3 -d reviews_shuffled.txt part_
wc -l reviews_shuffled.txt
```

Download → unzip → extract JSON text → clean HTML → lowercase → shuffle → shard → sanity-check the line count – eight commands from today's talk, chained into one real workflow.

Quick Reference: All 75 Commands

Closing – keep this page handy

- 1. **Navigation:** pwd, ls, cd, mkdir, rmdir, tree, find
- 2. **File Ops:** touch, cp, mv, rm, ln, file, stat, locate
- 3. **Disk & Perms:** du, df, chmod, chown, sudo, whoami
- 4. **Downloading:** wget, curl, ssh, scp, rsync, ping, git clone

- 5. **Compression:** tar, zip, unzip, gzip, gunzip
- 6. **Viewing:** cat, less, head, tail, wc, diff
- 7. **Text Processing:** grep, sed, awk, sort, uniq, cut, tr, paste, join, rev
- 8. **Pipes & Shell:** echo, printf, xargs, tee, history, alias, man, watch

- 9. **Data Prep:** split, shuf, comm, nl, iconv, dos2unix, md5sum, sha256sum, jq
- 10. **Monitoring:** ps, top, htop, kill, free, nvidia-smi
- 11. **Environment:** pip, conda, git status

Where to Go Next

Closing – keep learning

Today covered 75 commands in 90 minutes – fast, on purpose. Real fluency comes from *using* these on your own projects, not from memorising slides.

Practice ideas

- Redo every example on this deck against your own dataset this week
- Pick 5 commands and add them to your muscle memory before moving on
- Try chaining 3+ commands with pipes `|` to solve one real task

Further reading

- `man command_name` – the built-in, always-available reference
- Hostinger, “*60+ Basic Linux Commands*” (tutorial article)
- TecMint, “*40+ Linux Commands for Every ML Engineer*”
- explainshell.com – paste any command to see it explained

Thank You!

Now go open a terminal and try them yourself.

Ye Kyaw Thu

Lab Leader, Language Understanding Lab, Myanmar
Visiting Professor, Core-AI Lab, NECTEC, Thailand

<https://sites.google.com/site/yekyawthunlp/>

<https://github.com/ye-kyaw-thu>